

T2_Unit - 5_PQ/QB_VHA

263

Write Python code to train a kNN classifier using the following steps:

- Split the dataset X into training and testing sets with a test size of 0.3 and a random state of 42.
- Initialize a kNN classifier with 5 neighbors.
- Train the classifier on the training set.
- Make predictions on the test set.
- Calculate and print the accuracy score of the classifier.

```
In [1]: import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the Wine dataset
wine = load_wine()
X = wine.data
y = wine.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize a kNN classifier with 5 neighbors
knn = KNeighborsClassifier(n_neighbors=5)

# Train the classifier on the training set
knn.fit(X_train, y_train)

# Make predictions on the test set
y_pred = knn.predict(X_test)

# Calculate and print the accuracy score of the classifier
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of kNN classifier: {accuracy:.2f}')
```

Accuracy of kNN classifier: 0.74

264

Write Python code to train a decision tree classifier with entropy as the criterion using the following steps:

- Initialize a Decision Tree classifier with entropy as the criterion.
- Train the classifier on the training set.
- Make predictions on the test set.
- Calculate and print the confusion matrix for the classifier.

In [2]:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

# Load the Wine dataset
wine = load_wine()
X = wine.data
y = wine.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_st

# Initialize a Decision Tree classifier with entropy as the criterion
dt_classifier = DecisionTreeClassifier(criterion='entropy', random_state=42)

# Train the classifier on the training set
dt_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = dt_classifier.predict(X_test)

# Calculate and print the confusion matrix for the classifier
conf_matrix = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(conf_matrix)
```

Confusion Matrix:

```
[[18  1  0]
 [ 2 17  2]
 [ 2  1 11]]
```

265

Write Python code to evaluate the performance of a classification model using the following steps:

- Import the necessary functions from sklearn.metrics.
- Calculate and print the classification report for the true labels and predicted labels.
- Calculate and print the accuracy score of the classifier.

In [3]:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matri

# Load the Wine dataset
wine = load_wine()
X = wine.data
y = wine.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_st

# Initialize a Decision Tree classifier with entropy as the criterion
```

```

dt_classifier = DecisionTreeClassifier(criterion='entropy', random_state=42)

# Train the classifier on the training set
dt_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = dt_classifier.predict(X_test)

# Calculate and print the confusion matrix for the classifier
conf_matrix = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(conf_matrix)

# Import the necessary functions from sklearn.metrics
from sklearn.metrics import classification_report, accuracy_score

# Calculate and print the classification report for the true labels and predicted
class_report = classification_report(y_test, y_pred)
print('Classification Report:')
print(class_report)

# Calculate and print the accuracy score of the classifier
acc_score = accuracy_score(y_test, y_pred)
print('Accuracy Score:')
print(acc_score)

```

Confusion Matrix:

```

[[18  1  0]
 [ 2 17  2]
 [ 2  1 11]]

```

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.95	0.88	19
1	0.89	0.81	0.85	21
2	0.85	0.79	0.81	14
accuracy			0.85	54
macro avg	0.85	0.85	0.85	54
weighted avg	0.86	0.85	0.85	54

Accuracy Score:

0.8518518518518519

266

Using the Iris dataset ([Iris.csv](#)), write Python code to perform the following tasks:

- Split the dataset into features (X) and labels (y).
- Split the features and labels into training and testing sets with a test size of 0.2 and a random state of 42.
- Initialize a kNN classifier with 3 neighbors.
- Train the classifier on the training set.
- Make predictions on the test set.
- Calculate and print the accuracy score of the classifier.

```

In [4]: from sklearn.datasets import load_iris
        from sklearn.model_selection import train_test_split
        from sklearn.neighbors import KNeighborsClassifier

```

```

from sklearn.metrics import accuracy_score

# Load the Iris dataset from sklearn
iris = load_iris()
X = iris.data
y = iris.target

# Split the features and labels into training and testing sets with a test size of
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

# Initialize a kNN classifier with 3 neighbors
knn_classifier = KNeighborsClassifier(n_neighbors=3)

# Train the classifier on the training set
knn_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = knn_classifier.predict(X_test)

# Calculate and print the accuracy score of the classifier
acc_score = accuracy_score(y_test, y_pred)
print('Accuracy Score of kNN Classifier:', acc_score)

```

Accuracy Score of kNN Classifier: 1.0

267

You are tasked with using the k-Nearest Neighbors (kNN) algorithm to classify whether patients have diabetes or not based on certain diagnostic measurements. You have been provided with diabetes.csv file. The datasets consist of several medical predictor (independent) variables and one target (dependent) variable, Outcome. Independent variables include the number of pregnancies the patient has had, their BMI, insulin level, age, and so on. Also perform Model Performance Analysis using confusion matrix.

In [5]:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# Load the dataset
df = pd.read_csv('diabetes.csv')

# Display the first few rows of the dataset
print(df.head())

# Splitting the dataset into independent variables (X) and the target variable (y)
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

# Initializing the k-Nearest Neighbors classifier
knn = KNeighborsClassifier(n_neighbors=5)

# Fitting the classifier to the training data
knn.fit(X_train, y_train)

```

```
# Making predictions on the testing data
y_pred = knn.predict(X_test)

# Calculating the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Generating the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Accuracy: 0.6623376623376623

Confusion Matrix:

```
[[70 29]
 [23 32]]
```

268

The objective is to perform classification on the Iris dataset using the k-Nearest Neighbors (kNN) algorithm. The Iris dataset contains measurements of various iris flowers, including features such as sepal length, sepal width, petal length, and petal width, along with the corresponding species label. The problem involves two main tasks:

- Build a kNN classification model to predict the species of iris flowers based on their feature measurements.
- Train the model on a portion of the dataset and evaluate its performance on another portion to assess its accuracy.
- Experiment with different values of k and choose the optimal value that maximizes the model's performance.
- Use appropriate evaluation confusion matrix to evaluate the model's performance. Also calculate accuracy, sensitivity, and specificity.

Use iris.csv file for dataset.

```
In [6]: from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# Load the Iris dataset
iris = load_iris()
X = iris.data
```

```

y = iris.target

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

# Initializing the k-Nearest Neighbors classifier
k_values = [3, 5, 7, 9] # Experiment with different values of k
best_accuracy = 0
best_k = 0

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_k = k

print(f"Best k: {best_k}")
print(f"Best accuracy: {best_accuracy}")

# Using the best k to train the final model
knn_final = KNeighborsClassifier(n_neighbors=best_k)
knn_final.fit(X_train, y_train)

# Making predictions on the testing data
y_pred_final = knn_final.predict(X_test)

# Generating the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred_final)
print("Confusion Matrix:")
print(conf_matrix)

# Calculating accuracy, sensitivity, and specificity
total = conf_matrix.sum()
accuracy = (conf_matrix[0, 0] + conf_matrix[1, 1] + conf_matrix[2, 2]) / total
sensitivity = conf_matrix[0, 0] / (conf_matrix[0, 0] + conf_matrix[0, 1] + conf_ma
specificity = (conf_matrix[1, 1] + conf_matrix[1, 2] + conf_matrix[2, 1] + conf_ma
               conf_matrix[1, 0] + conf_matrix[1, 1] + conf_matrix[2, 0] + conf_matri

print(f"Accuracy: {accuracy}")
print(f"Sensitivity: {sensitivity}")
print(f"Specificity: {specificity}")

```

```

Best k: 3
Best accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Accuracy: 1.0
Sensitivity: 1.0
Specificity: 1.0

```

268

Given the Breast Cancer Wisconsin (Diagnostic) dataset, the objective is to build a kNN classification model that accurately predicts whether a tumor is benign or malignant based on the diagnostic features provided. The model should be trained

on a portion of the dataset and evaluated on another portion to assess its performance. The ultimate goal is to create a reliable classifier that can assist healthcare professionals in diagnosing breast cancer accurately and early. Use cancer.csv file for dataset.

```
In [7]: # Import necessary Libraries
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load the Breast Cancer Wisconsin dataset
data = load_breast_cancer()

# Create a DataFrame from the dataset
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

# Split the dataset into features (X) and target (y)
X = df.drop('target', axis=1)
y = df['target']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize the kNN classifier
k = 5 # Number of neighbors
knn = KNeighborsClassifier(n_neighbors=k)

# Train the kNN classifier
knn.fit(X_train_scaled, y_train)

# Predict the labels for test set
y_pred = knn.predict(X_test_scaled)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Accuracy: 0.9590643274853801

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.94	0.94	63
1	0.96	0.97	0.97	108
accuracy			0.96	171
macro avg	0.96	0.95	0.96	171
weighted avg	0.96	0.96	0.96	171

Confusion Matrix:

```
[[ 59  4]
 [  3 105]]
```

269

Given the credit card transaction dataset, the objective is to build a kNN classification model that accurately predicts whether a transaction is fraudulent or non-fraudulent based on the transaction features provided. The model should be trained on historical transaction data and evaluated on another portion of the dataset to assess its performance. The ultimate goal is to create a reliable classifier that can automatically detect fraudulent transactions and prevent financial losses for credit card companies and cardholders. Use card_transdata.csv for dataset.

```
In [10]: # Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load the dataset
data = pd.read_csv("card_transdat.csv")

# Assuming the target variable is named 'Class'
X = data.drop('fraud', axis=1)
y = data['fraud']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_st

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize the kNN classifier
k = 5 # Number of neighbors
knn = KNeighborsClassifier(n_neighbors=k)

# Train the kNN classifier
knn.fit(X_train_scaled, y_train)

# Predict the labels for test set
y_pred = knn.predict(X_test_scaled)
```

```
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Accuracy: 0.99869

Classification Report:

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	273871
1.0	0.99	0.99	0.99	26129
accuracy			1.00	300000
macro avg	1.00	0.99	1.00	300000
weighted avg	1.00	1.00	1.00	300000

Confusion Matrix:

```
[[273741  130]
 [  263 25866]]
```

270

The task involves building a k-Nearest Neighbors (kNN) regression model to predict the Air Quality Index (AQI) based on the latitude and longitude coordinates of various countries. The dataset used for this task contains information about the AQI levels and geographic locations (latitude and longitude) of different countries. The AQI serves as an indicator of air quality, with higher values indicating poorer air quality and vice versa. Use AQI and Lat Long of Countries.csv for dataset.

```
In [11]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Step 1: Load the dataset
data = pd.read_csv('AQI and Lat Long of Countries.csv')

# Step 2: Feature Selection
X = data[['lat', 'lng']]
y = data['AQI Value']

# Step 3: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

# Step 5: Building the kNN Model
k = 5 # Number of neighbors
knn_regressor = KNeighborsRegressor(n_neighbors=k)
knn_regressor.fit(X_train, y_train)

# Step 6: Model Evaluation
```

```

y_pred = knn_regressor.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Absolute Error:", mae)
print("Mean Squared Error:", mse)
print("R-squared:", r2)

# Step 8: Prediction
# Example of new latitude and longitude coordinates
new_data = pd.DataFrame([[40.7128, -74.0060], # New York City
                        [34.0522, -118.2437], # Los Angeles
                        [51.5074, -0.1278]], # London
                        columns=['lat', 'lng'])

# Making predictions
new_predictions = knn_regressor.predict(new_data)

print("Predicted AQI for New York City:", new_predictions[0])
print("Predicted AQI for Los Angeles:", new_predictions[1])
print("Predicted AQI for London:", new_predictions[2])

```

Mean Absolute Error: 16.818388739143455
Mean Squared Error: 818.7628990715783
R-squared: 0.5063279996943211
Predicted AQI for New York City: 68.2
Predicted AQI for Los Angeles: 112.4
Predicted AQI for London: 55.2

271

The task involves building a Decision Tree classifier to predict whether to play tennis based on weather conditions. The dataset used for this task is the PlayTennis dataset, which contains information about various weather attributes such as outlook, temperature, humidity, and wind, along with the corresponding decision to play tennis or not. Use PlayTennis.csv for dataset.

```

In [12]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import OneHotEncoder

# Step 1: Load the dataset
data = pd.read_csv('PlayTennis.csv')

# Step 2: Data Preprocessing
# Perform one-hot encoding for categorical variables
data = pd.get_dummies(data, columns=['outlook', 'temp', 'humidity', 'windy'])

# Step 3: Split the dataset into features and target variable
X = data.drop('play', axis=1)
y = data['play']

# Step 4: Building the Decision Tree Classifier
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Step 5: Model Evaluation

```

```

y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Step 6: Visualization (Optional)
# Decision tree visualization is omitted here as it's not directly supported with

# Step 7: Prediction
# Example of new data for prediction
new_data = pd.DataFrame({'outlook': ['rainy'], 'temp': ['mild'], 'humidity': ['nor
# Perform one-hot encoding for new data
new_data = pd.get_dummies(new_data)
# Add missing columns to match training data
missing_cols = set(X.columns) - set(new_data.columns)
for col in missing_cols:
    new_data[col] = 0
new_data = new_data[X.columns] # Reorder columns to match X
prediction = clf.predict(new_data)
print("Predicted decision to play tennis:", prediction[0])

```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
no	1.00	1.00	1.00	1
yes	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Predicted decision to play tennis: no

272

Imagine that you are a medical researcher compiling data for a study. You have collected data about a set of patients, all of whom suffered from the same illness. During their course of treatment, each patient responded to one of 5 medications, Drug A, Drug B, Drug C, Drug X, and Drug Y. Part of your job is to build a model to find out which drug might be appropriate for a future patient with the same illness. The feature sets of this dataset are Age, Sex, Blood Pressure, and Cholesterol of patients, and the target is the drug that each patient responded to.

- It is a sample of multiclass classifier, and you can use the training part of the dataset to build a decision tree, and then use it to predict the class of a unknown patient, or to prescribe it to a new patient. Use drug200.csv for dataset.

In [13]:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load the dataset
data = pd.read_csv('drug200.csv')

# Step 2: Data Preprocessing

```

```

# No missing values or outliers handling is included in this example.

# Step 3: Feature Engineering
# Perform one-hot encoding for categorical variables
data = pd.get_dummies(data, columns=['Sex', 'BP', 'Cholesterol'])

# Step 4: Split the dataset into features and target variable
X = data.drop('Drug', axis=1)
y = data['Drug']

# Step 5: Building the Decision Tree Classifier
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Step 6: Model Evaluation
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Step 7: Prediction
# Example of new data for prediction
new_data = pd.DataFrame({'Age': [50], 'Sex': ['F'], 'BP': ['LOW'], 'Cholesterol': ['H']})
# Perform one-hot encoding for new data
new_data = pd.get_dummies(new_data, columns=['Sex', 'BP', 'Cholesterol'])
# Add missing columns to match training data
missing_cols = set(X.columns) - set(new_data.columns)
for col in missing_cols:
    new_data[col] = 0
new_data = new_data[X.columns] # Reorder columns to match X
# Predict the appropriate drug for the new patient
prediction = clf.predict(new_data)
print("Predicted drug for the new patient:", prediction[0])

```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
drugA	1.00	1.00	1.00	6
drugB	1.00	1.00	1.00	3
drugC	1.00	1.00	1.00	5
drugX	1.00	1.00	1.00	11
drugY	1.00	1.00	1.00	15
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

Predicted drug for the new patient: drugX

273 "You have been provided with a dataset named students_performance.csv, which contains various features related to students' performance in exams. The dataset includes features such as gender,

race/ethnicity, parental level of education, lunch, test preparation course, and scores in math, reading, and writing. Your task is to perform classification to predict whether a student will pass or fail based on these features.

Steps to complete:

1. Load the Dataset:

- o Load the dataset students_performance.csv into a pandas DataFrame.
- o Display the first few rows to understand the structure of the dataset and check the statistics.

2. Preprocessing:

o Convert the scores in math, reading, and writing to a binary pass/fail label. Consider a score of 50 or above as pass(1) and below 50 as fail(0).

- o Drop the scores in math, reading, and writing.
- o Create a new Column named 'Overall_Pass' and create a binary label: pass(1) if Student passes all heads (math, reading, and writing) and fail(0) if student has failed in even one head.
- o Encode categorical variables (gender, race/ethnicity, parental level of education, lunch, test preparation course) using one-hot encoding.
- o Handle any missing values if present.

o Split the dataset into features (x) and target (y) where the target is the pass/fail label for the math pass. o Split the data into training (80%) sets using random state 10.

```
In [15]: # =====
# Import Required Libraries
# =====
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# =====
# 1. Load Dataset
# =====
data = pd.read_csv("StudentsPerformance.csv")

# =====
# 2. Preprocessing
```

```

# =====
# Convert scores into pass/fail
data['math_pass'] = np.where(data['math score'] >= 50, 1, 0)
data['reading_pass'] = np.where(data['reading score'] >= 50, 1, 0)
data['writing_pass'] = np.where(data['writing score'] >= 50, 1, 0)

# Overall pass
data['Overall_Pass'] = np.where(
    (data['math_pass'] == 1) &
    (data['reading_pass'] == 1) &
    (data['writing_pass'] == 1),
    1, 0
)

# Drop original score columns
data = data.drop(['math score', 'reading score', 'writing score'], axis=1)

# One-hot encoding
categorical_cols = [
    'gender',
    'race/ethnicity',
    'parental level of education',
    'lunch',
    'test preparation course'
]

data = pd.get_dummies(data, columns=categorical_cols, drop_first=True)

# Handle missing values
data = data.dropna()

# =====
# Define Features and Target
# =====
X = data.drop('math_pass', axis=1)
y = data['math_pass']

# =====
# Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=10
)

# =====
# Decision Tree Model
# =====
dt = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=4,
    random_state=42
)

dt.fit(X_train, y_train)

# =====
# Prediction
# =====
y_pred = dt.predict(X_test)

```

```

# =====
# Evaluation
# =====
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Extract TP, TN, FP, FN
TP = cm[1,1]
TN = cm[0,0]
FP = cm[0,1]
FN = cm[1,0]

print("\nTP:", TP, "TN:", TN, "FP:", FP, "FN:", FN)

# Additional Metrics
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)

print("\nPrecision:", precision)
print("Recall (Sensitivity):", recall)
print("Specificity:", specificity)

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

```

Accuracy: 0.955

Confusion Matrix:

```
[[ 15   7]
 [  2 176]]
```

TP: 176 TN: 15 FP: 7 FN: 2

Precision: 0.9617486338797814

Recall (Sensitivity): 0.9887640449438202

Specificity: 0.6818181818181818

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.68	0.77	22
1	0.96	0.99	0.98	178
accuracy			0.95	200
macro avg	0.92	0.84	0.87	200
weighted avg	0.95	0.95	0.95	200

274"Write Python code to train a Decision Tree Classifier using the following steps:

- a) Assume a sample dataset is already loaded and corresponding features and labels are already defined in the environment.
- b) Split the dataset into training and testing sets using 75% of the data for training and set random state of 10.
- c) Initialize an entropy-based Decision Tree Classifier with maximum depth of 3. Use this model to make predictions on the test data.
- d) Print confusion matrix and accuracy score of the classifier."

```
In [ ]: # Import required libraries
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# Assume X (features) and y (labels) are already defined

# Step (b): Split dataset (75% training, random_state = 10)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=10
)

# Step (c): Initialize Decision Tree (entropy, max_depth = 3)
dt_model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=3,
    random_state=10
)

# Train the model
dt_model.fit(X_train, y_train)

# Make predictions
y_pred = dt_model.predict(X_test)

# Step (d): Evaluate the model
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

# Print results
print("Confusion Matrix:")
print(cm)

print("\nAccuracy Score:")
print(accuracy)
```

275 " Breast Cancer Detection using SVM

Using the cancer.csv dataset, build a classification model to predict whether a tumor is benign or malignant.

Perform the following:

Load and preprocess dataset. Separate features and target. Split dataset into training and testing sets. Train SVM classifier. Predict test dataset. Evaluate model accuracy."

```
In [16]: # Import Libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 1: Load dataset
data = load_breast_cancer()

# Step 2: Separate features and target
X = data.data
y = data.target # 0 = malignant, 1 = benign

# Step 3: Split dataset (75% training, 25% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=10
)

# Step 4: Train SVM classifier (without scaling)
svm_model = SVC(kernel='rbf', random_state=10)
svm_model.fit(X_train, y_train)

# Step 5: Predict test data
y_pred = svm_model.predict(X_test)

# Step 6: Evaluate model
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

# Output results
print("Confusion Matrix:")
print(cm)

print("\nAccuracy Score:")
print(accuracy)
```

Confusion Matrix:

```
[[44  8]
 [ 3 88]]
```

Accuracy Score:

```
0.9230769230769231
```

311 " Diabetes Prediction using Random Forest

You are given a dataset diabetes.csv containing medical records of patients. The dataset includes predictor variables such as BMI, Age, Insulin, Pregnancies, etc., and a target variable named Outcome.

Your task is to:

Load the dataset using pandas.

Separate independent and dependent variables. Split dataset into training and testing sets. Train a Random Forest Classifier. Predict patient outcomes.

Evaluate performance using:

Confusion Matrix

Accuracy Score"

```
In [17]: # Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# Step 1: Load dataset
data = pd.read_csv("diabetes.csv")

# Step 2: Separate independent (X) and dependent (y) variables
X = data.drop("Outcome", axis=1)
y = data["Outcome"]

# Step 3: Split dataset (75% training, 25% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=10
)

# Step 4: Train Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=10)
rf_model.fit(X_train, y_train)

# Step 5: Predict outcomes
y_pred = rf_model.predict(X_test)

# Step 6: Evaluate model
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

# Output results
print("Confusion Matrix:")
print(cm)

print("\nAccuracy Score:")
print(accuracy)
```

Confusion Matrix:

```
[[109  12]
 [ 33  38]]
```

Accuracy Score:

```
0.765625
```

312 "Iris Dataset Classification using SVM

Using the Iris dataset, perform classification using Support Vector Machine by completing the following steps:

Load the dataset. Separate features (X) and target labels (y).

Split dataset into: Training set (80%) Testing set (20%) random_state = 42 Train an SVM classifier with C = 1. Predict the test dataset labels. Print the model accuracy."

```
In [18]: # Import required Libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Step 1: Load dataset
iris = load_iris()

# Step 2: Separate features and target
X = iris.data
y = iris.target

# Step 3: Split dataset (80% training, 20% testing, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Train SVM classifier (C = 1)
svm_model = SVC(C=1)
svm_model.fit(X_train, y_train)

# Step 5: Predict test data
y_pred = svm_model.predict(X_test)

# Step 6: Print accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
```

Model Accuracy: 1.0

313 "Random Forest Classifier Implementation

Write Python code to implement a Random Forest Classifier using the following instructions: Initialize the classifier with: n_estimators = 100 random_state = 42 Train the model using training data. Predict class labels for test data.

Evaluate the model by calculating: Confusion Matrix Accuracy score"

```
In [20]: # Import required Libraries
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# Assume X_train, X_test, y_train, y_test are already defined

# Step 1: Initialize Random Forest Classifier
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Step 2: Train the model
```

```

rf_model.fit(X_train, y_train)

# Step 3: Predict test data
y_pred = rf_model.predict(X_test)

# Step 4: Evaluate the model
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

# Print results
print("Confusion Matrix:")
print(cm)

print("\nAccuracy Score:")
print(accuracy)

```

Confusion Matrix:

```

[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

```

Accuracy Score:

1.0

314 "SVM Model Training and Evaluation

Write a Python program to implement a Support Vector Machine (SVM) classifier using the following steps: Split the dataset into training and testing sets using: Test size = 0.3
Random state = 42 Initialize the SVM classifier with: C = 1.0 (Regularization parameter)
Train the model using the training dataset. Predict the class labels for the test dataset.

Calculate and print: Accuracy score Classification report"

```

In [19]: # Import required libraries
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Assume X (features) and y (target) are already defined

# Step 1: Split dataset (70% training, 30% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Step 2: Initialize SVM classifier
svm_model = SVC(C=1.0)

# Step 3: Train the model
svm_model.fit(X_train, y_train)

# Step 4: Predict test data
y_pred = svm_model.predict(X_test)

# Step 5: Evaluate model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

```

```
# Output results
print("Accuracy Score:")
print(accuracy)

print("\nClassification Report:")
print(report)
```

Accuracy Score:
1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

In [14]:

```
import numpy as np
from sklearn.metrics import confusion_matrix

def calculate_metrics(y_true, y_pred):
    """
    Calculates confusion matrix and various classification metrics.

    Args:
        y_true (np.ndarray): Ground truth labels.
        y_pred (np.ndarray): Predicted labels.

    Returns:
        dict: A dictionary containing the confusion matrix and calculated metrics
              (accuracy, error, specificity, sensitivity, recall, precision, f2_score)
    """
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()
    print("tn, fp, fn, tp")
    print(tn, fp, fn, tp)

    accuracy = (tp + tn) / (tp + tn + fp + fn)
    error = 1 - accuracy
    specificity = tn / (tn + fp) if (tn + fp) != 0 else np.nan
    sensitivity = tp / (tp + fn) if (tp + fn) != 0 else np.nan
    recall = sensitivity
    precision = tp / (tp + fp) if (tp + fp) != 0 else np.nan
    f2_score = (5 * precision * recall) / (4 * precision + recall) if (4 * precision + recall) != 0 else np.nan

    metrics = {
        "confusion_matrix": cm,
        "accuracy": accuracy,
        "error": error,
        "specificity": specificity,
        "sensitivity": sensitivity,
        "recall": recall,
        "precision": precision,
        "f2_score": f2_score,
    }
    return metrics
```

```

if __name__ == '__main__':
    # Example usage:
    y_true = np.array([1, 0, 1, 0, 1, 1, 0, 0, 1, 0])
    y_pred = np.array([1, 1, 1, 0, 0, 1, 0, 1, 1, 0])

    results = calculate_metrics(y_true, y_pred)

    print("Confusion Matrix:")
    print()
    print(results["confusion_matrix"])
    print("\nMetrics:")
    print(f"Accuracy: {results['accuracy']:.4f}")
    print(f"Error: {results['error']:.4f}")
    print(f"Specificity: {results['specificity']:.4f}")
    print(f"Sensitivity (Recall): {results['sensitivity']:.4f}")
    print(f"Recall: {results['recall']:.4f}")
    print(f"Precision: {results['precision']:.4f}")
    print(f"F2 Score: {results['f2_score']:.4f}")

```

tn, fp, fn, tp

3 2 1 4

Confusion Matrix:

```
[[3 2]
 [1 4]]
```

Metrics:

Accuracy: 0.7000

Error: 0.3000

Specificity: 0.6000

Sensitivity (Recall): 0.8000

Recall: 0.8000

Precision: 0.6667

F2 Score: 0.7692

In [7]:

```

import numpy as np
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, classification_report
y_true = np.array([1, 0, 1, 0, 1, 1, 0, 0, 1, 0])
y_pred = np.array([1, 1, 1, 0, 0, 1, 0, 1, 1, 0])
print(confusion_matrix(y_true, y_pred))
print(accuracy_score(y_true, y_pred))
print(precision_score(y_true, y_pred))
print(classification_report(y_true, y_pred))

```

```
[[3 2]
```

```
[1 4]]
```

```
0.7
```

```
0.6666666666666666
```

	precision	recall	f1-score	support
0	0.75	0.60	0.67	5
1	0.67	0.80	0.73	5
accuracy			0.70	10
macro avg	0.71	0.70	0.70	10
weighted avg	0.71	0.70	0.70	10

In [22]:

```

import numpy as np
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, classification_report
y_true = np.array(['cat', 'dog', 'cat', 'dog', 'cat', 'cat', 'dog', 'dog', 'cat', 'cat'])
y_pred = np.array(['cat', 'cat', 'cat', 'dog', 'dog', 'cat', 'dog', 'cat', 'cat', 'cat'])

```

```
print(confusion_matrix(y_true,y_pred,labels=["cat","dog"]))
print(accuracy_score(y_true,y_pred))
print(precision_score(y_true,y_pred,labels=["cat","dog"],pos_label="cat"))
print(classification_report(y_true,y_pred))
```

```
[[4 1]
 [2 3]]
0.7
0.6666666666666666
```

	precision	recall	f1-score	support
cat	0.67	0.80	0.73	5
dog	0.75	0.60	0.67	5
accuracy			0.70	10
macro avg	0.71	0.70	0.70	10
weighted avg	0.71	0.70	0.70	10

In [23]:

```
import numpy as np
from sklearn.metrics import confusion_matrix,accuracy_score,precision_score,classification_report
y_true = np.array(['cat', 'dog', 'cat', 'dog', 'cat', 'cat', 'dog', 'dog', 'cat', 'cat'])
y_pred = np.array(['cat', 'cat', 'cat', 'dog', 'dog', 'cat', 'dog', 'cat', 'cat', 'cat'])

print(confusion_matrix(y_true,y_pred,labels=["dog","cat"]))
print(accuracy_score(y_true,y_pred))
print(precision_score(y_true,y_pred,labels=["cat","dog"],pos_label="cat"))
print(classification_report(y_true,y_pred))
```

```
[[3 2]
 [1 4]]
0.7
0.6666666666666666
```

	precision	recall	f1-score	support
cat	0.67	0.80	0.73	5
dog	0.75	0.60	0.67	5
accuracy			0.70	10
macro avg	0.71	0.70	0.70	10
weighted avg	0.71	0.70	0.70	10